ate

Transport Layer (TCP)

TCP is a Point-to-Point, connection-oriented, reliable transfer protocol. It builds directly on the concepts like ACK's, checksum, timers, cumulative ACK's, sliding windows, but adapts them for real world byte-stream delivery.

Characteristics:

1) Point-to point:
Communication between one sender & receiver only. (multicasting not possible).

2) Reliable in-order byte stream:
TCP treats data as continuous stream of bytes.

3) Full duplex Data Flow:
Bi-directional data flow. (Both sender & receiver can send request & response over same transfer).

BHATTI

CamScanner

date

4) Connection oriented:

3 way handshake.

5) Pipelined:

Uses pipelined protocols. & slt window size.

6) Flow control:

Sender will not overwhelm receiver. (It will know receivers buffered capacity & not send too much data).

- Remember TCP is transport layer so only lives in end devices not in routers or switches

BHATTI

day/date

TCP connection & data sending:

Once the handshake is done, 2 app processes can send data to each other.

A TCP connection has:

- Buffers
- Sockets connection.

Consider we are sending data from client to server:

1) Client process passes a stream of data through the socket.
2) After this socket, data is in hands of client's TCP.
3) This data is directed to the send buffer.
4) Then from time to time TCP will grab chunks of data from send buffer (segments) & pass the to network layer.

* PDU of Transport layer is "segment"

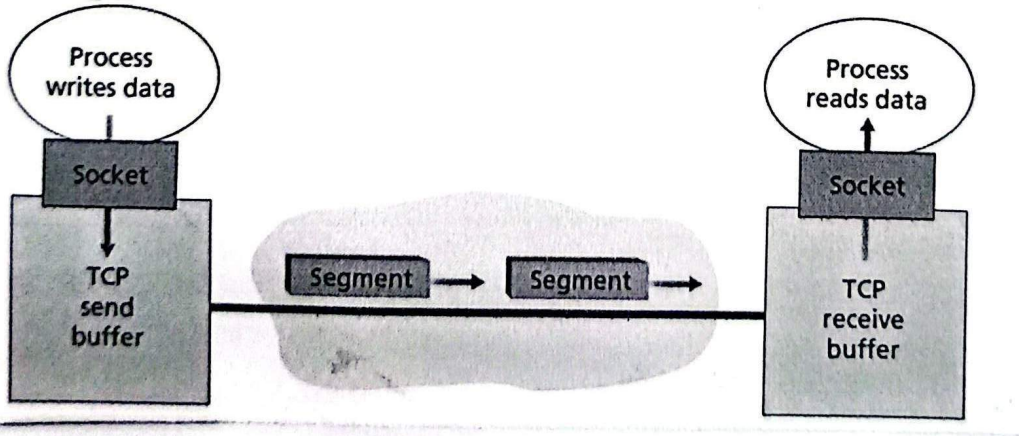
BHATTI

CamScanner

"TCP sends data at its own convenience."

Maximum amount of data in segment:
This is limited by the (MSS) maximum Segment Size.

The MSS is found & set by first determining the length of the largest link-layer frame that can be sent by the local sending host — MTU (maximum transmission Unit).
After knowing MTU is the MSS set ensuring that the full TCP segment (header plus data) can fit in the MTU. plus the network headers.



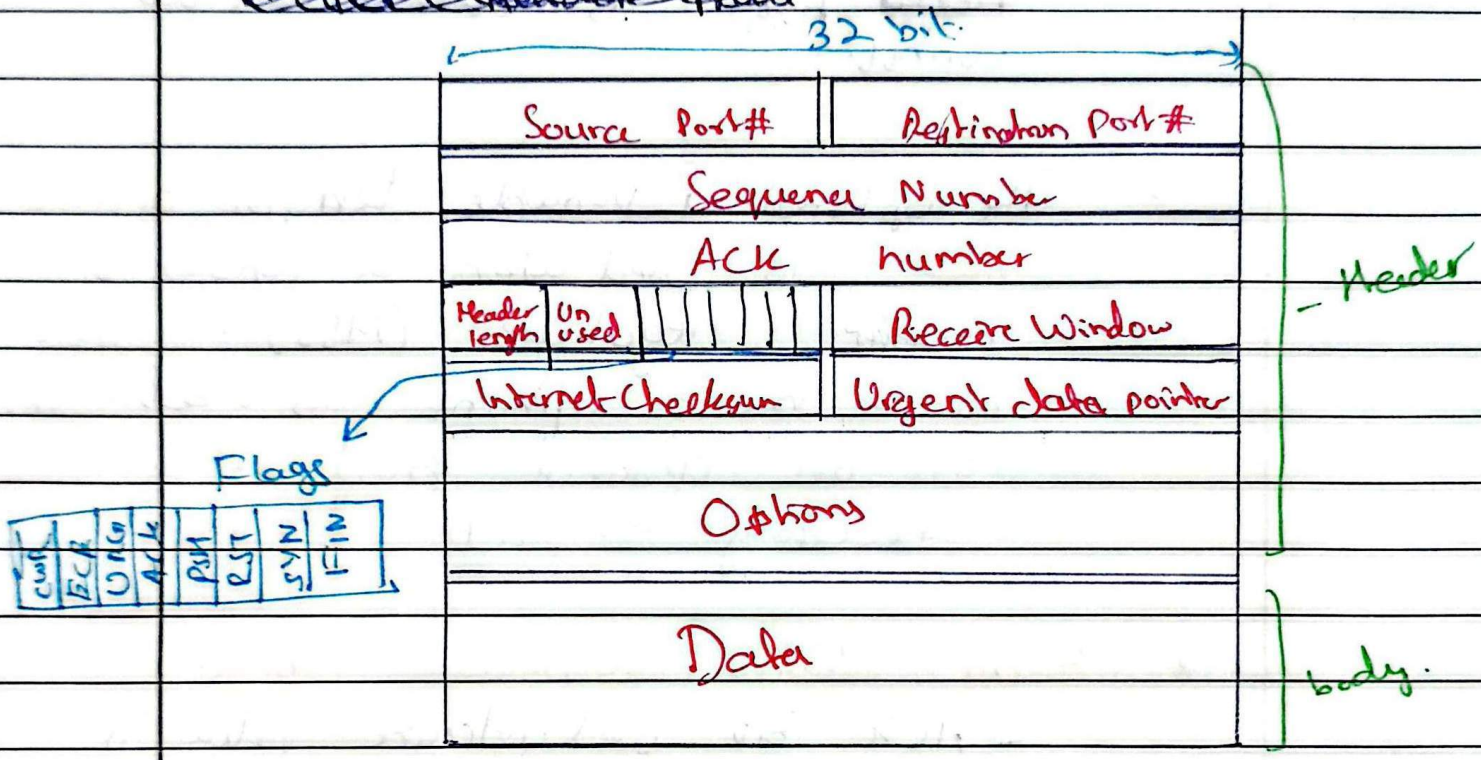
BHATTI

day/date

TCP segment structure:

A TCP segment structure is typically a minimum 20 bytes. (can be more as well)

~~A TCP header field~~



Details:

* Like UDP, TCP includes Ports, checksum.
* The 32-bit Seq Number & ACK number are used by sender & receiver in implementing reliable data transfer

BHATTI

day/date

* The 4-bit header length field specifies the length of TCP header. in 32 bit words.

* The 16-bit receive window field. is used for flow control.

* The optional & variable length options field is used when a sender & receiver negotiate (MSS). There can be more options as well. This is an optional field, not necessary for TCP.

* Flag field containing 6 bits.
  - ACK bit used indicate value in ACK number is valid.
  - RST, SYN & FIN used for connection setup & teardown
  - The CWR & ECE bits are used in explicit congestion notifications.
  - Setting PSH means receiver should pass the data to upper layer immediately.

RST = reset
SYN = sync
FIN = Finish
PSH = Push

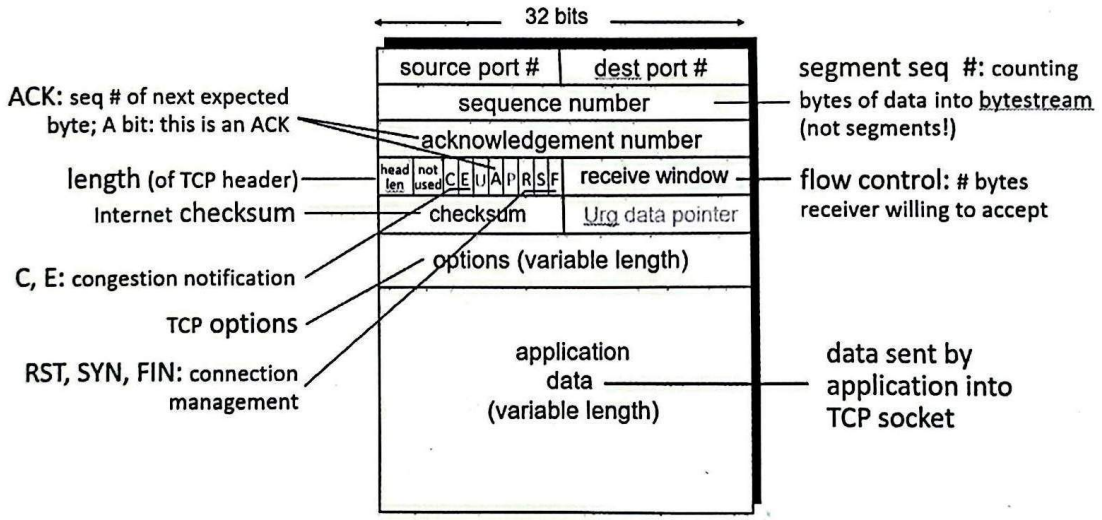
BHATTI

day/date

- URG bit is used to indicate that there is data in this segment that is urgent to send & given priority.

It is used with the urgent data Pointer field (16-bits). which tells the location of last byte ~~Used~~ of this urgent data.

* Fun fact:- In practically PSH, URG & urgent data pointer are never used but are mentioned for completeness.



BHATTI

CamScanner

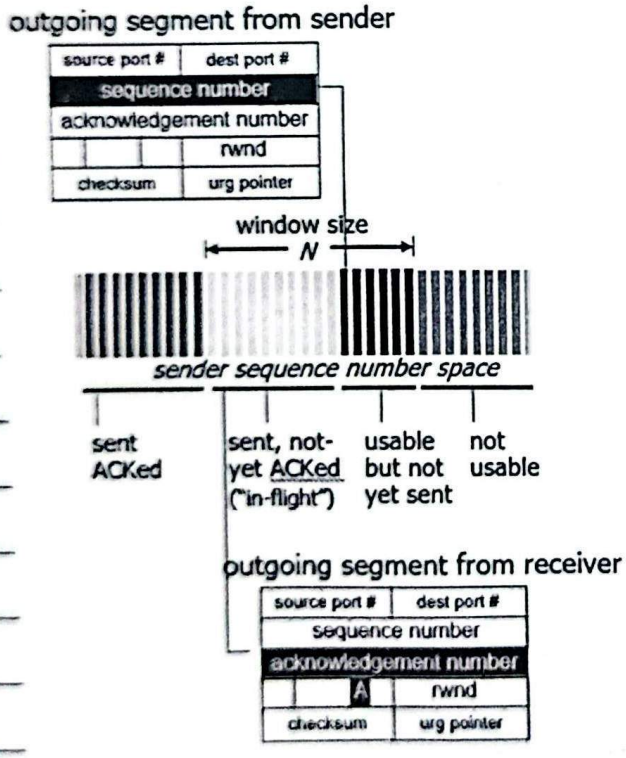
day/date

TCP sequence Numbers & ACK numbers

Unlike RDT protocols that use count packets as sequence Numbers, TCP sequence Numbers count bytes in the data stream.

The ACKnowledgements sent by receiver of the next ~~expected~~ expected byte.
- Cumulative ACK used.

In rdt the ACK was of the same as seq. but in TCP ACK is of the next expected byte



BHATTI

date

Example:

Consider a process in Host A wants to send a stream of data to host B.

File = 500,000 bytes.
MSS = 1000 bytes., first byte of data stream is numbered 0.

So total segments = $\frac{500000}{1000} = 500$

Segment 1 = 0 - 999 bytes.

Segment 2 = 1000 - 1999 bytes.

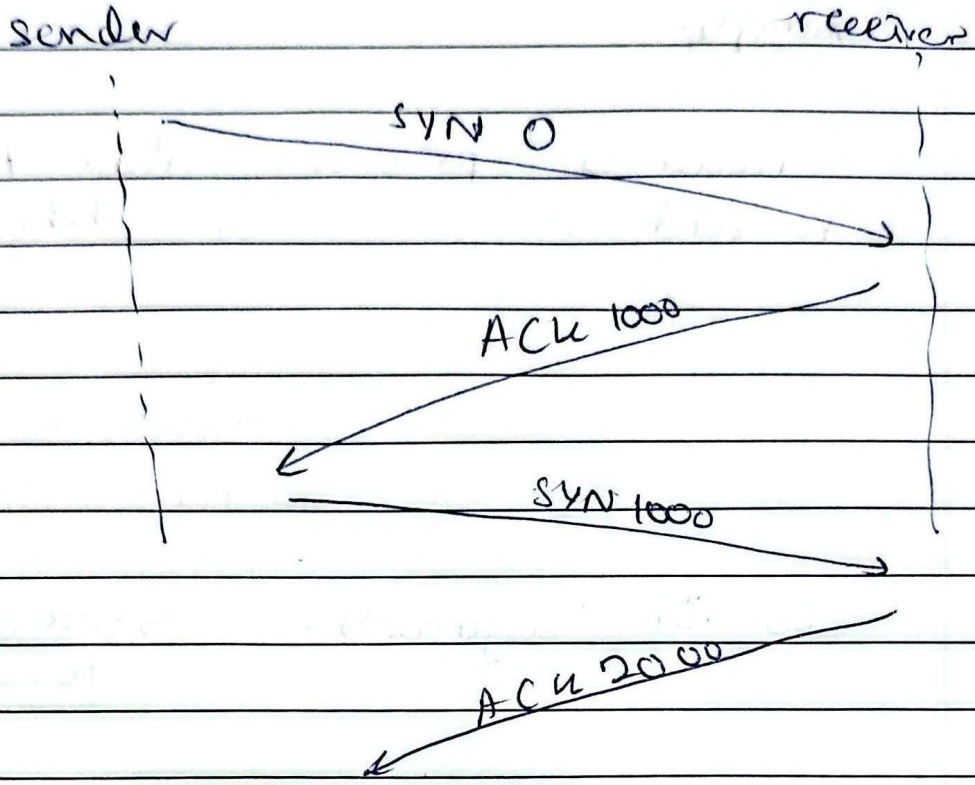
how our segment 1, seq num = 0.
segment 2, seq number = 1000.
first byte of each otehrs seqeunce number.

~~order~~

BHATTI

CamScanner

day/date



Our first ACK is 1000 which is the first ~~data~~ byte of our 2nd segment. so ACK is the next required byte.

BHATTI

y/date

Full duplex

Our last example is half complete because remember that TCP is full duplex.

Host A might be receiving some data from B while it sends data to B.

The ACK in Host A is the sequence number that it expect next from Host B. while the sequence number in Host A is the ~~dat~~ bit byte of the data it is sending.

Consider an example:

~~Host A received all byte numbers~~ to 535 from B

~~Host A is waiting for byte 536.~~

~~etc~~

~~that's all~~

BHATTI

CamScanner

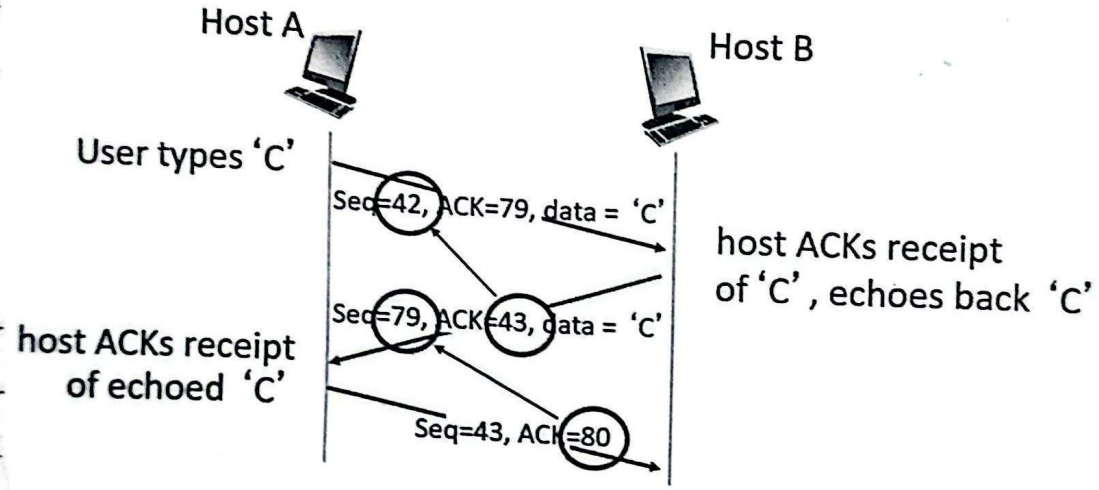
day/date

- Host A has received all bytes numbered 0 through 535 from B.
- Host A is waiting for byte 536 and all the subsequent bytes in Host B's data stream.

- What does Host A put in the acknowledgment number field of the next segment it sends to B?

Host A puts 536 in the acknowledgment number field of the segment it sends to B

Another example: (imp. cumulative ACK):



TCP provides Cumulative ACK's.

BHATTI

y/date

Out of order Segments:

- TCP only ACK's bytes up to the first missing byte in stream (cumulative ACK).

- For out of order segments, it can either discard them or buffer them (Buffering is actually used in practice).

Random Initial Sequence numbers:

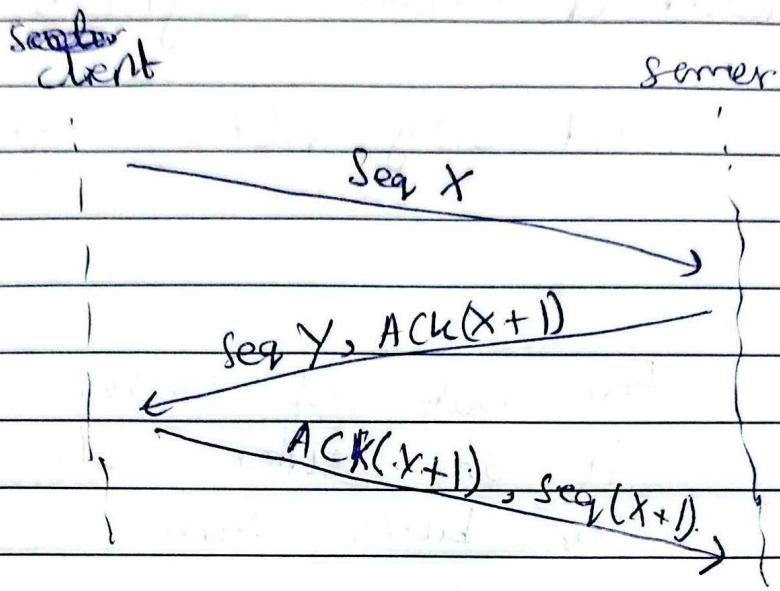
When a TCP connection is established during 3 way handshake, the initial Sequence Number is not 0. (we were doing that for the examples).

Instead both start with a random 32-bit numbers.

CamScanner

y/date

e.g.:



X, Y are random numbers.

After intial handshake: all data bytes are counted relative to these intial values eg first segement will have Seq X+1.

Why use Random Intial Sequence:

1) Prevent TCP prediction attack:
If all number start from 0, attacking it is easy.

BHATTI

late

2) Preventing Inter-connection confusion:

Data packets can sometime linger inside the router due to network congestion.

This might become an old ghost packet that might suddenly arrive in an the process transmission.

Now if all Packets started from seq 0, this packet would have caused data corruption.

But randomness has a huge chance to prevent it because the ghost packets seq will be different & not in window of transfer.

Remember

* Relative sequence numbers are used e.g. (X+1)' not absolute.

but growing software might show absolutes like 0, 1000, 2000, because they do the subtraction math.

BHATTI

day/date

Example:

- Host A has received all bytes numbered 0 through 535 from B.
- Host A has also received another segment containing bytes 900 through 1,000
  - For some reason Host A has not yet received bytes 536 through 899.
- What does Host A put in the acknowledgment number field of the next segment it sends to B?

A's next segment to B will contain 536 in the acknowledgment number field

BHATTI

CamScanner

e

Round Trip Time estimation & Timeout:

TCP like rdt uses timeout/retransmit mechanism to recover from lost segments.

Although conceptually it is simple, it actually has many problems.

Estimating RTT:

we need to set a value for RTT but if we make it:

~~Too short~~

Too short: premature timeouts.

Too long: slow reaction to segment loss.

so how to estimate:

1) First of all a Sample RTT is taken.
Now this Sample RTT is time between when segment is sent & when an Ack is received

BHATTI

day/date

Also instead of taking / measuring a new sample RTT for every segment. The Sample RTT is charged approximately once every RTT.

Also TCP never computes a sample RTT for a segment that has never been transmitted.

But we cannot just use sample RTT, because it has huge fluctuations. so it is better to take some average RTT.

2) we take Estimated RTT. This is what is used in TCP.

Estimated RTT = $(1-\alpha)$* Estimated RTT + α*SampleRTT

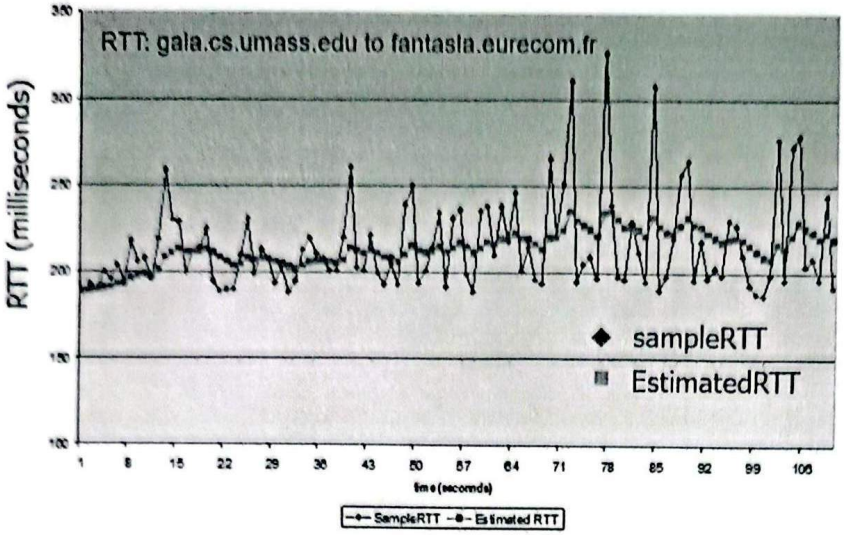
new estimate RTT

previous / current Estimated RTT

α = constant typically 0.125

BHATTI

day/da



3) But with this it is also needed to find the deviation of sample RTT from estimated RTT because we need to account for the fact that network might be noisy.

$$DevRTT_{new} = (1-\beta) * DevRTT + \beta * (SampleRTT - EstimatedRTT)$$

β = constant = typically 0.25.

BHATTI

y/date

4) Total timeout interval:

Timeout interval = Estimated RTT + 4*DevRTT

Time it would normally take

To account for congestion, noise in network. ACTS as "Safety margin".

IMP:

* Initial Timeout Interval value of 1 second is recommended by RFC.

* (Imp) what happens when a packet is loss & timeout happens:

If a segment times out, TCP doubles its timer & applies Exponential back-off.

how for the segments retransmitted:

$$Timeout_{new} = 2 * Timeout_{old}$$

The time keeps on doubling every time packet loss or timeout happens so exponentially grows.

BUT this stops as soon as Ack for new data arrives, TCP exits back off mode & recalculates the interval. using the standard formula.

It does backoff, because persistent timeout signal heavy congestion & we don't want to add more & more packets again & again which leads to more congestion

BHATTI

day/date

TCP : reliable data transfer:

we know that the Internet's network protocol is unreliable.

TCP creates a reliable data transfer service on top of IP's unreliable best effort service.

what TCP ~~do~~ does:

- Pipelined segments
- cumulative ACK's
- Single transmission timer

- Retransmissions by:
  - timeout
  - duplicate ACK

BHATTI

CamScanner

TCP Sender (simplified)

event: data received from application

- create segment with seq #
- seq # is byte-stream number of first data byte in segment
- start timer if not already running
  - think of timer as for oldest unACKed segment
  - expiration interval: `TimeOutInterval`

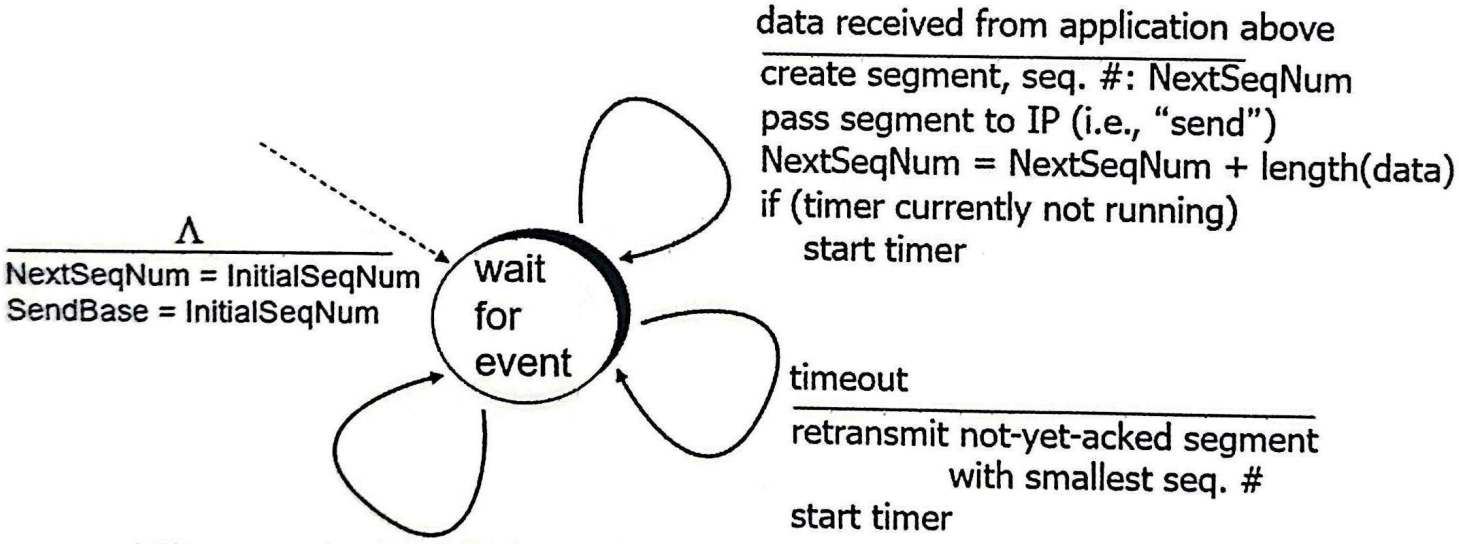
event: timeout

- retransmit segment that caused timeout
- restart timer

event: ACK received

- if ACK acknowledges previously unACKed segments
  - update what is known to be ACKed
  - start timer if there are still unACKed segments

TCP sender (simplified)

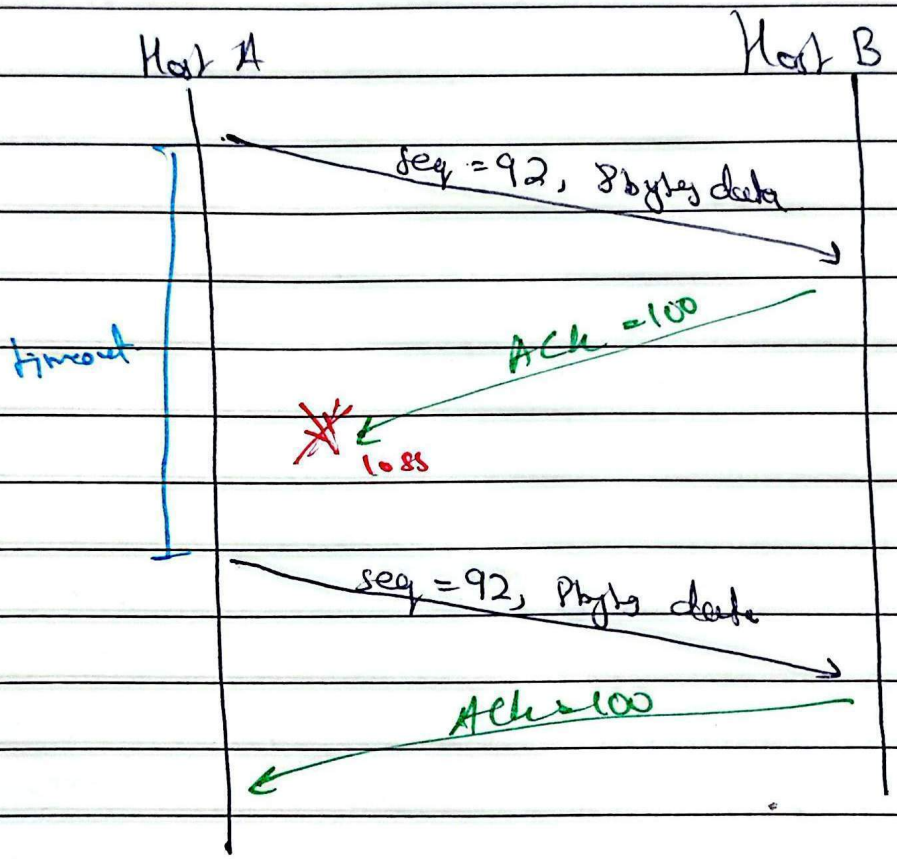


CamScanner

/date

Normal TCP retransmissions :

1) lost ACK:



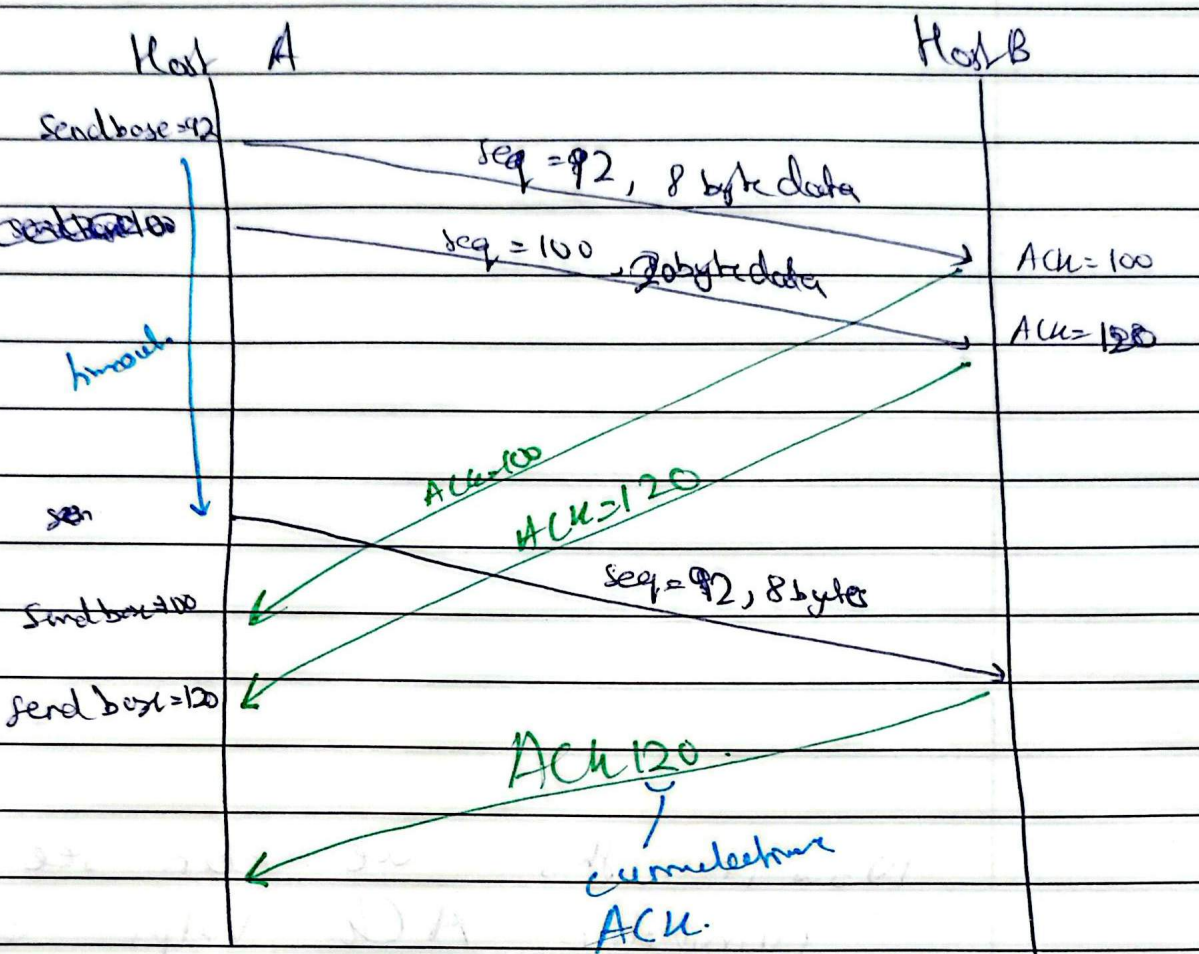
we calculated our ACK seq = 100 by 92+8 = 100. So we ~~see~~. next required byte was 100 th byte, which became ACK.

BHATTI

CamScanner

ate

2) Premature timeout.



ACK 120.

cumulative ACK.

as Host B was waiting not by 120, it asked for this it already has 100 - 119.

* Send base ~~does~~ changes when ACK comes only.

BHATTI

day/date

3) Cumulative ACK:

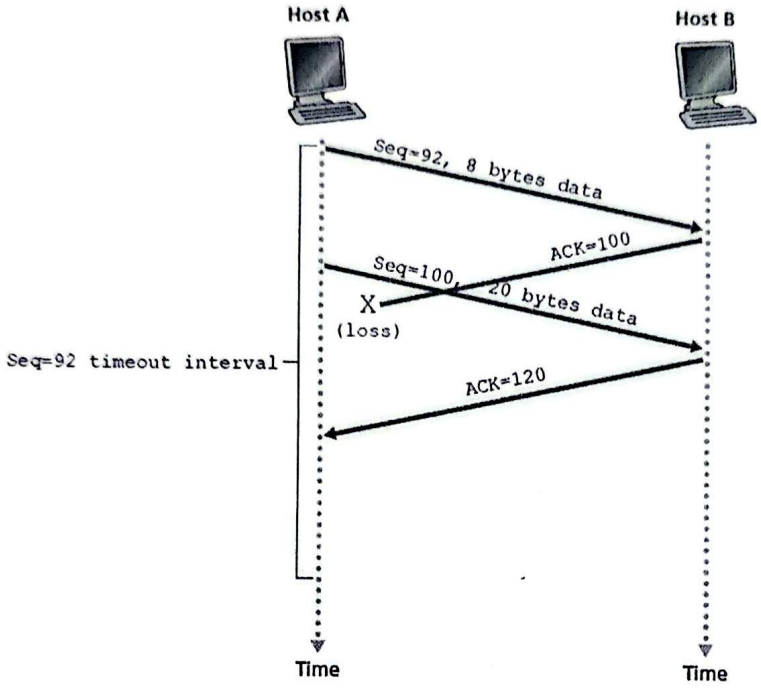


Figure 3.36 ♦ A cumulative acknowledgment avoids retransmission of the first segment

Now in this we can see how cumulative ACK helps even if ACK is lost. Also because our timer was bigger & the cumulative ACK reached first.

BHATTI

CamScanner

day/date

Now we will see some improvements in some Modified TCPs.

1) Modified TCP – Doubling Timeout Interval:

We have already discussed this before we studied TCP's rdt.

Every time timeout occurs, TCP retransmits the not-yet-acked segment with smallest sequence num. But each transmission gets timeout$_{old}$ * 2. (twice the previous timeout)

For example, suppose `TimeoutInterval` associated with the oldest not yet acknowledged segment is .75 sec when the timer first expires. TCP will then retransmit this segment and set the new expiration time to 1.5 sec. If the timer expires again 1.5 sec later, TCP will again retransmit this segment, now setting the expiration time to 3.0 sec. Thus, the intervals grow exponentially after each retransmission. However, whenever the timer is started after either of the two other events (that is, data received from application above, and ACK received), the `TimeoutInterval` is derived from the most recent values of `EstimatedRTT` and `DevRTT`.

BHATTI

CamScanner

day/date

2) Modified TCP – fast retransmit:

- This solves the problem that timeout transmissions can take too long.
- It does this by using the duplicate ACK's.

Concept:

If sender receiving 3 additional ACK's for same data (one original + 3 duplicate) resend un ACK'ed segment with smallest seq #.

The 3 duplicate show that likely the packet was lost so retransmit.

BHATTI

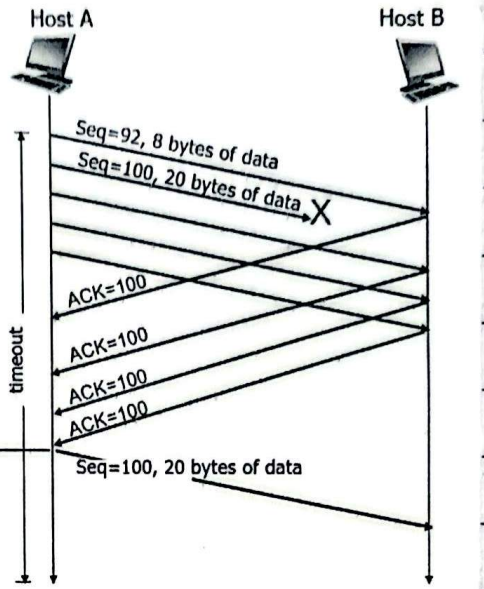
'date

TCP fast retransmit

if sender receives 3 additional ACKs for same data ("triple duplicate ACKs"), resend unACKed segment with smallest seq #

- likely that unACKed segment lost, so don't wait for timeout

Receipt of three duplicate ACKs indicates 3 segments received after a missing segment – lost segment is likely. So retransmit!



- If time out happens before 3 duplicate come then retransmit & ignore the 3 dup.

Retransmission happens if either of the 2 happens first.

BHATTI

TCP Receiver: ACK generation [RFC 5681]

Event at receiver	*TCP receiver action*
arrival of in-order segment with expected seq #. All data up to expected seq # already ACKed	delayed ACK. Wait up to 500ms for next segment. If no next segment, send ACK
arrival of in-order segment with expected seq #. One other segment has ACK pending	immediately send single cumulative ACK, ACKing both in-order segments
arrival of out-of-order segment higher-than-expect seq. # . Gap detected	immediately send *duplicate ACK*, indicating seq. # of next expected byte
arrival of segment that partially or completely fills gap	immediate send ACK, provided that segment starts at lower end of gap

CamScanner

TCP Flow control:

TCP : GB-N or Selective repeat:

Let us close our study of TCP's error-recovery mechanism by considering the following question: Is TCP a GBN or an SR protocol? Recall that TCP acknowledgments are cumulative and correctly received but out-of-order segments are not individually ACKed by the receiver. Consequently, as shown in Figure 3.33 (see also Figure 3.19), the TCP sender need only maintain the smallest sequence number of a transmitted but unacknowledged byte (`SendBase`) and the sequence number of the next byte to be sent (`NextSeqNum`). In this sense, TCP looks a lot like a GBN-style protocol. But there are some striking differences between TCP and Go-Back-N. Many TCP implementations will buffer correctly received but out-of-order segments [Stevens 1994]. Consider also what happens when the sender sends a sequence of segments $1, 2, \ldots, N$, and all of the segments arrive in order without error at the receiver. Further suppose that the acknowledgment for packet $n < N$ gets lost, but the remaining $N - 1$ acknowledgments arrive at the sender before their respective timeouts. In this example, GBN would retransmit not only packet n, but also all of the subsequent packets $n + 1, n + 2, \ldots, N$. TCP, on the other hand, would retransmit at most one segment, namely, segment n. Moreover, TCP would not even retransmit segment n if the acknowledgment for segment $n + 1$ arrived before the timeout for segment n.

A proposed modification to TCP, the so-called **selective acknowledgment** [RFC 2018], allows a TCP receiver to acknowledge out-of-order segments selectively rather than just cumulatively acknowledging the last correctly received, in-order segment. When combined with selective retransmission—skipping the retransmission of segments that have already been selectively acknowledged by the receiver—TCP looks a lot like our generic SR protocol. Thus, TCP's error-recovery mechanism is probably best categorized as a hybrid of GBN and SR protocols.

BHATTI

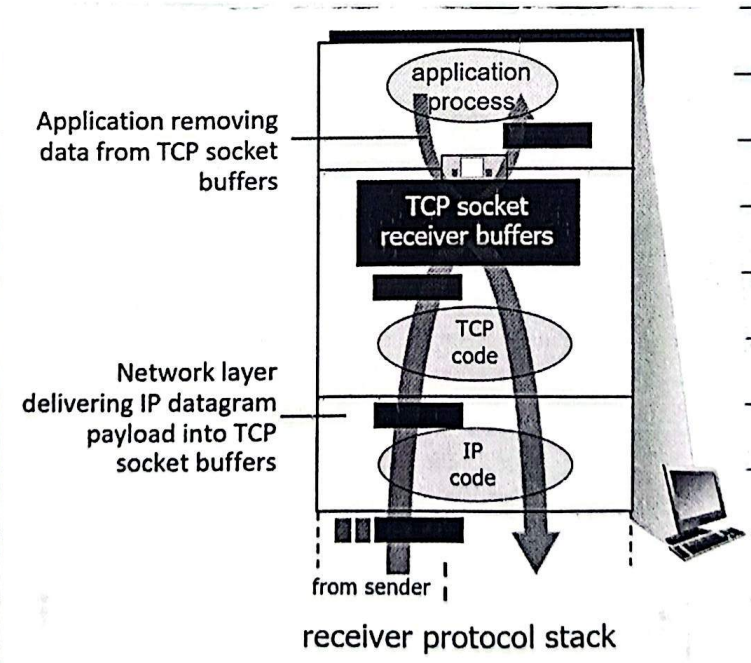
CamScanner

day/date

TCP: Flow control:

What happens if network layer deliver data faster than app layer removes from socket buffers?

In receiver side, if network keeps on filling the TCP buffers but app layer is slow to get data, the buffer would fill up! So how to fix that?



BHATTI

CamScanner

ate

To fix this, TCP provides: Flow control.

Remember that in TCP segment there is a 16-bit receive window field.

that shows how many bytes is the receiver willing to accept!

This field allows receiver to control sender so sender won't overflow the receiver's buffer. by transmitting too much & fast.

receive window = rwnd (short form):

How is rwnd calculated:

rwnd = RcvBuffer - [last Byte rcvd - last byte read].

Size of RCV Buffer

last Byte received

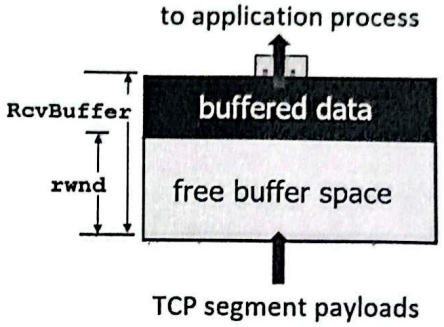
number of last byte in data stream saved in buffer of process in B.

last byte read by the receiver from its buffer.

BHATTI

lay/date

- TCP receiver "advertises" free buffer space in **rwnd** field in TCP header
  - **RcvBuffer** size set via socket options (typical default is 4096 bytes)
  - many operating systems autoadjust **RcvBuffer**
- sender limits amount of unACKed ("in-flight") data to be received **rwnd**
- guarantees receive buffer will not overflow



TCP receiver-side buffering

* Remember: sender maintains the rwnd of receiver because it has to know how much to send.

BHATTI

CamScanner

ate

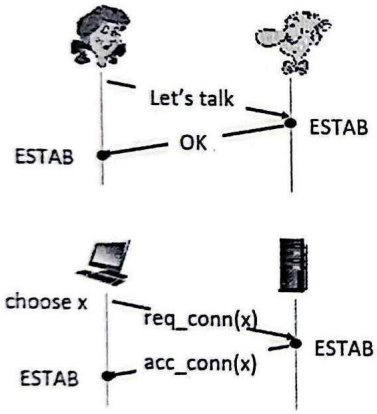
TCP connection management:

Before exchanging any data, sender & receiver handshake:

* agree to establish connection.
* agree on parameters like (seq #'s).

Considering a 2 way handshake:

2-way handshake:



Q: will 2-way handshake always work in network?

- variable delays
- retransmitted messages (e.g. req_conn(x)) due to message loss
- message reordering
- can't "see" other side

BHATTI

CamScanner

day/date

A 2 way - handshake is good but it can fail as well:

consider Host A send a request - connection packet to B but due to time out it resends it.

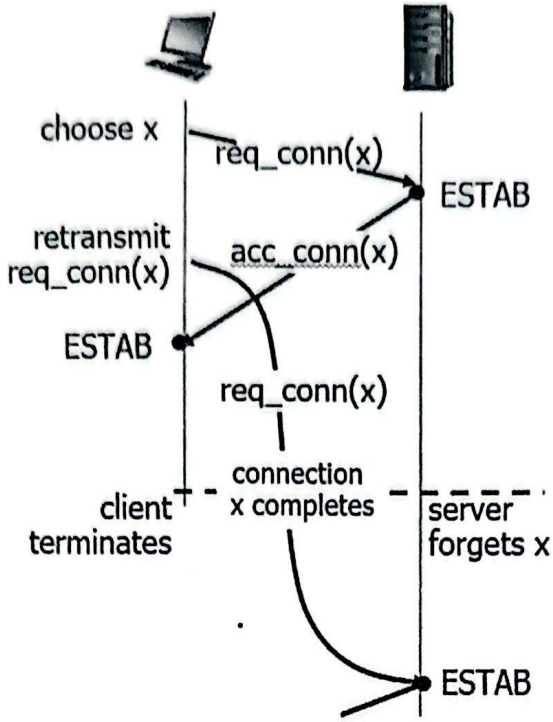
Now the first request - comm packet reached server but second got into a network traffic.

But connection is established & all data sharing happens & connection is closed.

But Now the second request of connection got out of network traffic & Host B establishes connection But Host A already forgot about her if got its work done.

It remains a half open connection.

BHATTI



Problem: half open connection! (no client)

So to fix this:

TCP has 3 way handshake.

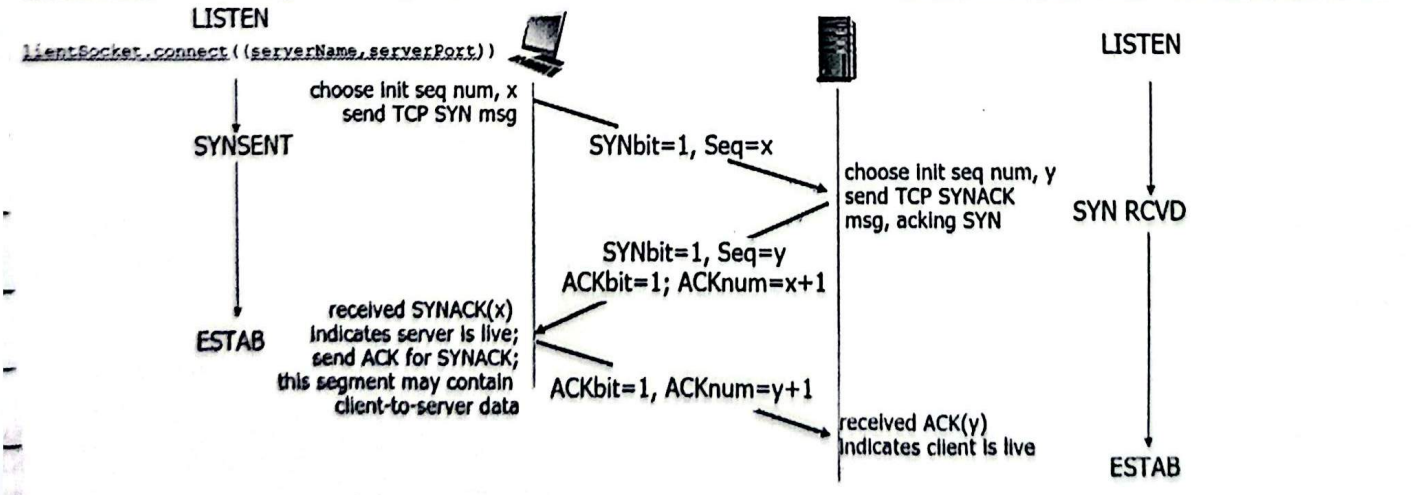
3 way - handshake:

Server state

```
serverSocket = socket(AF_INET,SOCK_STREAM)
serverSocket.bind(('',serverPort))
serverSocket.listen(1)
connectionSocket, addr = serverSocket.accept()
```

Client state

```
clientSocket = socket(AF_INET, SOCK_STREAM)
```



CamScanner

y/date

Step1): The TCP sends a special TCP segment to server side TCP. This segment has no app data but its SYN flag is set to 1.
This is called SYN segment.

Step2): Once TCP SYN arrives at server. Server allocates the TCP buffers & variables to the connection.
This then decides 3 thing: for segment.

a) SYN bit is set to 1.

b) ACK is set to client_initial sequence number + 1.

c) own SYN is set to by a random number server_isn (initial-sequence) number.

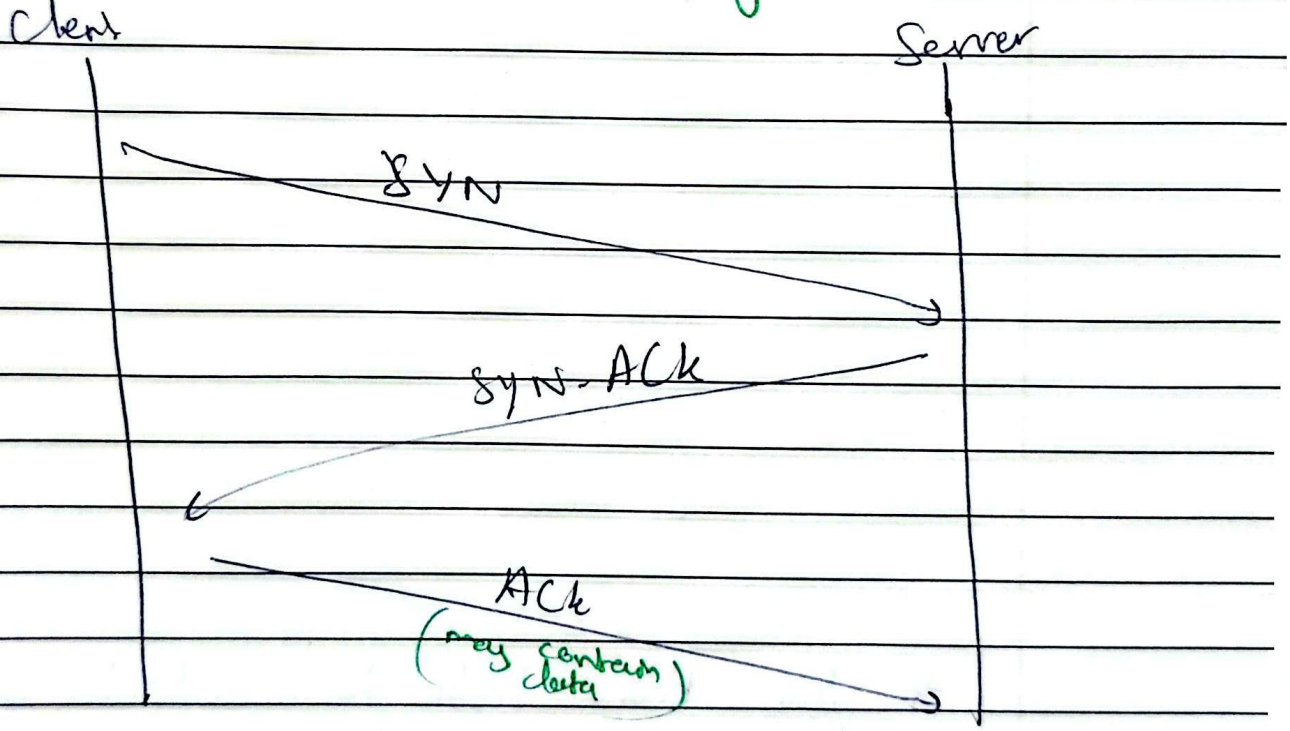
This is called SYN-ACK. Segment.

BHATTI

day/date

Step3): Upon receiving SYNACK, client itself allocates buffers & variables to connection.
& now this acknowledges the Servers connection granted segment by putting value server_isn+1 in the ACK segment.
SYN bit is now set to 0.

Now this last segment for connection can also contain data in payload.



BHATTI

CamScanner

day/date

Closing a TCP connection:

Then for closing a TCP connection, each server & client close their side of connection by setting FIN bit to 1 in TCP segment.

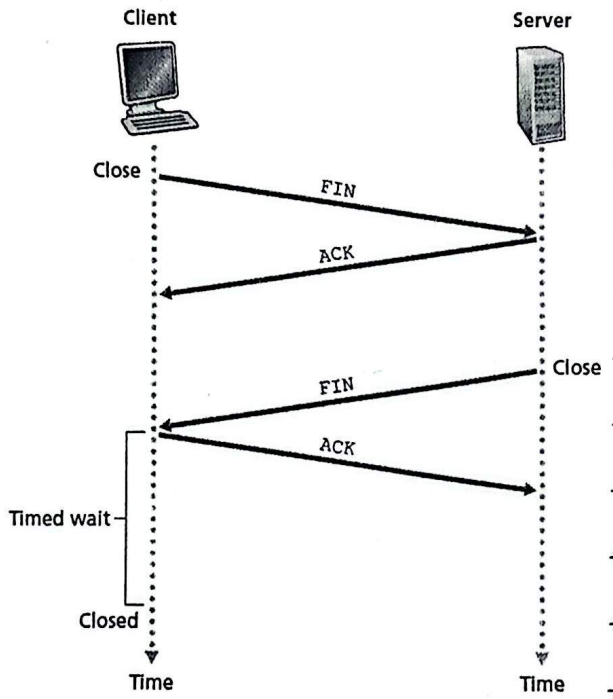


Figure 3.40 ♦ Closing a TCP connection

* This then leads to client & servers deallocation their buffers as well.

BHATTI

date

Principles of congestion Control:

"Too many sources sending toomuch data too fast for network to handle"

It is different from flow control.

Flow Control	Congestion Control
Receiver's buffer is overwhelmed by Sender.	Router's / network buffers are stopped from being overwhelmed by sender.
	• Router's buffers are limited.

Problems with congestion:

* Causes long delays.
* packet losses can occur (if Router buffer filled).

BHATTI

day/date

Now let's see simple scenerios

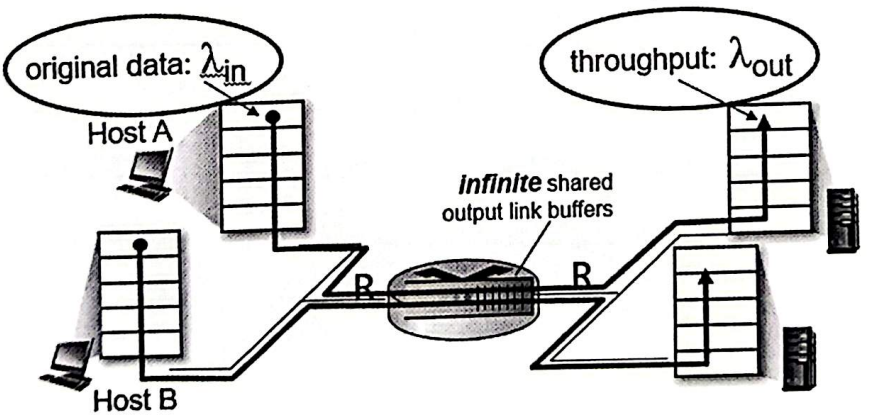
1) Scenerio1 : One router, infinite buffers.

Consider a simple scenerio where we have one router that has infinite space, now congestion can never happen.

A few terms:

λ_{in} : App layer sending rate.

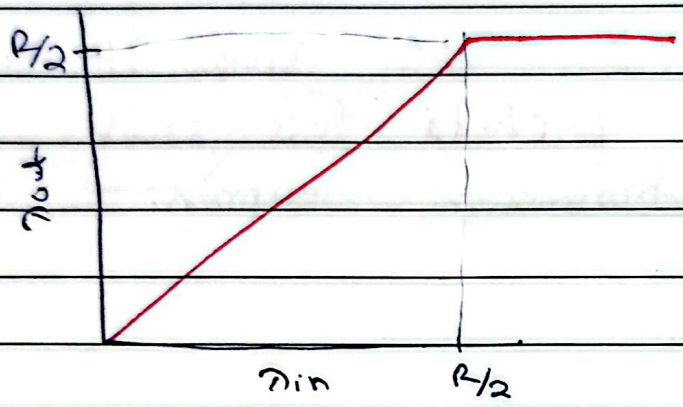
λ_{out} : Throughput (rate at which actually delivered).



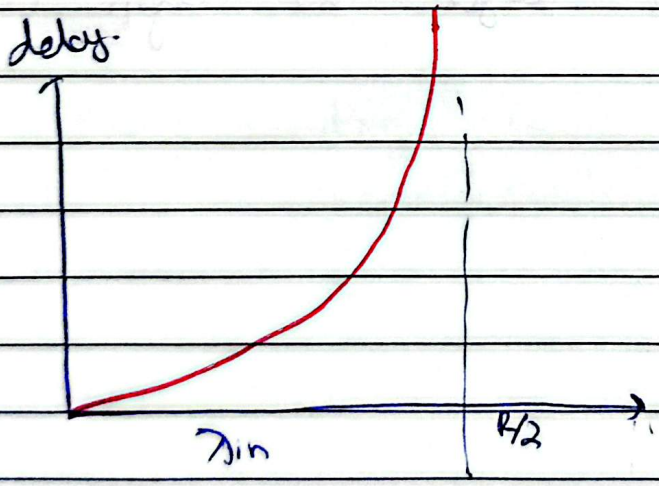
BHATTI

CamScanner

As there are 2 hosts so total throughput for each = $\frac{R}{2}$.



our λout is increasing with λin but becomes constant when we reach max throughput.



delay exponentially increases as λin reaches R/2. Packet arriving faster than they can be transmitted.

BHATTI

day/date

2) Scenario 2: one router finite buffers.

Now again we have a single router but buffers are finite.

Due to this if buffer of router filled. then packets loss starts happening so retransmissions happen.

New terms:

λ_{in} : original data
λ'_{in} : retransmitted + original data

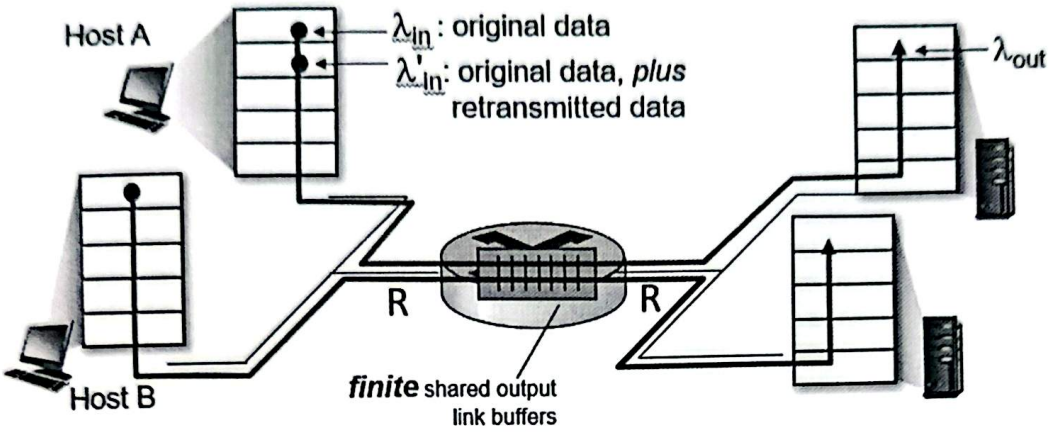
But remember λ'_{in} is only part of transport layer for app layer $\lambda_{in} = \lambda_{out}$

BHATTI

CamScanner

/date

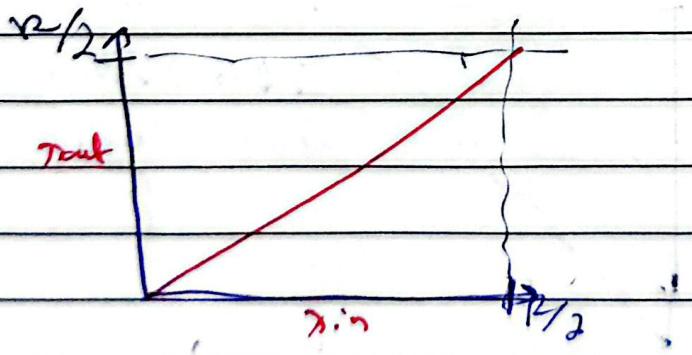
- sender retransmits lost, timed-out packet
  - application-layer input = application-layer output: $\lambda_{in} = \lambda_{out}$
  - transport-layer input includes *retransmissions* : $\lambda'_{in} \geq \lambda_{in}$



Now let's see how it works.

(a) consider Senders has Perfect knowledge of all router buffers.

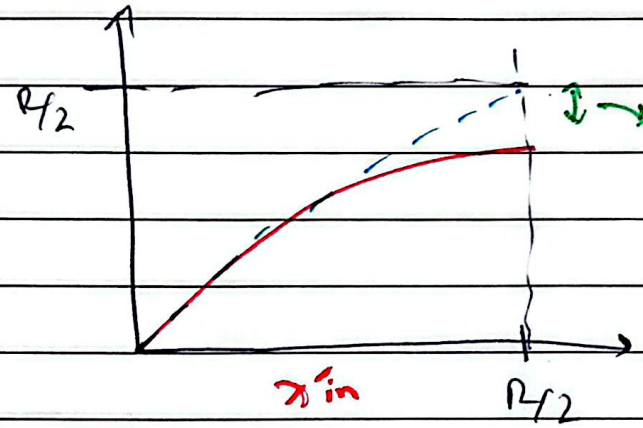
Now sender will only send with buffer is available.



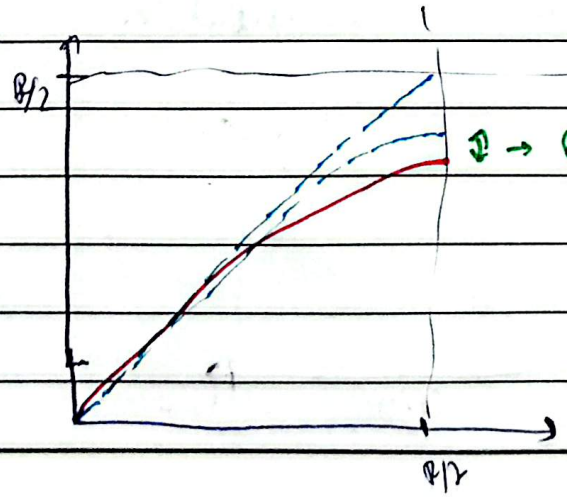
BHATTI

day/date

(b) But this not possible. So the sender can some knowledge like due to timer sender can know when a packet has been dropped.



(c) Now let's also include duplicate packets which are not needed e.g due to premature timeouts.



BHATTI

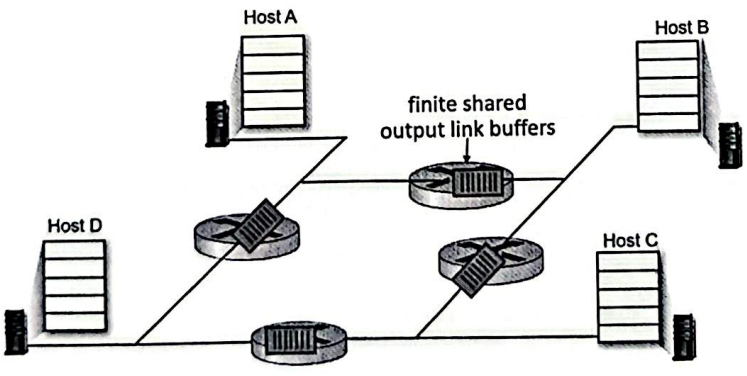
/date

So all costs of congestion in this scenario:
- more work for given receiver throughput
- unneeded transmissions. decreases overall max throughput.

3) Scenario 3: Multiple routers.

In reality there is not only one router but multiple routers within a network.

e.g consider:
- four senders
- multi hop paths.



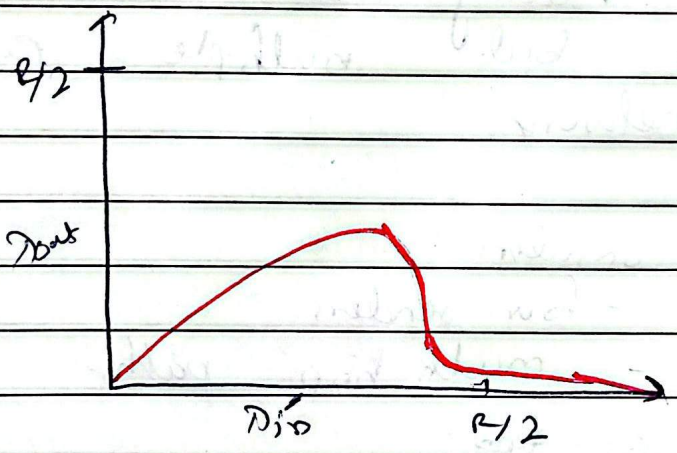
BHATTI

CamScanner

lay/date

Now seeing the previous image consider The Host A → C & B → D uses the same intermediate routers.

Now if Host A has alot of retransmissions then buffer of router fills up. & all pkts from B to D also are dropped.



BHATTI

CamScanner

ate

Costs of congestion: (all the insights):

* Throughput can never exceed capacity.

* Delay increases as capacity approached.

* un-needed duplicates further decreases effective throughput

* loss & retransmissions decreases effective throughput

* upstream transmission capacity / buffering wasted for packets loss.

BHATTI

CamScanner

//date

Now we understand how congestion can happen & its causes but how to control it.

2 Approaches of congestion control:

1) END * to end congestion control:

Sender & receiver handle congestion, there is no explicit feedback from network

- congestion inferred from observed loss, delay
- approach taken by TCP.

BHATTI

CamScanner

date

2) Network-assisted congestion control:

- routers provide direct feedback to sending/receiving hosts with flows passing through congested router
- may indicate congestion level or explicitly set sending rate.

BHATTI

CamScanner

day/date

TCP congestion Control: AIMD

"Additive Increase Multiplicative decrease" (AIMD)

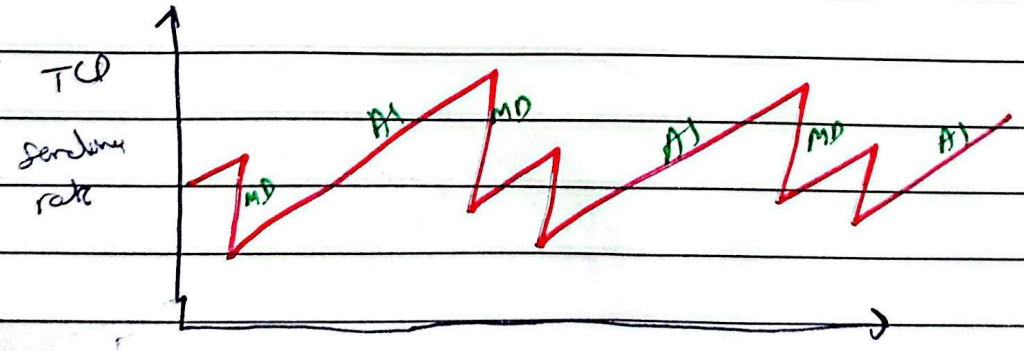
senders can increase sending rate until packet loss (Congestion occurs), then decrease sending rate on loss event

Additive Increase: (AI)

Increase sending rate by 1 MSS every RTT until loss detected.

Multiplicative decrease: (MD)

cut sending rate in half at each loss event.



BHATTI

ay/date

TCP Reno vs TCP Tahoe

TCP Reno:

It distinguishes between how we got to know about loss.

If Timeout, cuts window (cwnd) send rate (cwnd) to 1 MSS.

If 3 duplicate ACK's comes, cuts window size (cwnd) in half & enters fast retransmit.

Tcp Tahoe:

Treats all events equally.

Always cuts window size to 1 MSS & starts growing from there.

To break it down strictly:

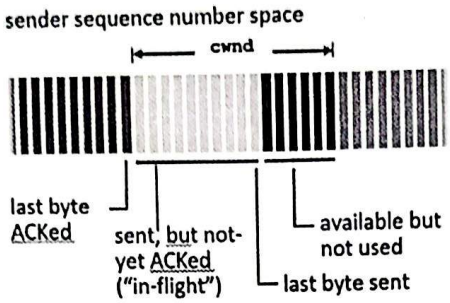
1. If the window is cut in half ($cwnd \approx cwnd/2$):
   - It is definitely TCP Reno (or another modern variant like CUBIC).
   - TCP Tahoe *never* cuts its window in half—it lacks the Fast Recovery state entirely and always drops `cwnd` down to 1 MSS on any loss event.
2. If the window drops to 1 MSS:
   - You cannot tell which protocol it is based solely on the window drop.
   - If it is TCP Tahoe, it drops to 1 MSS for *both* 3 Duplicate ACKs and Timeouts.
   - If it is TCP Reno, it *also* drops to 1 MSS when a Timeout occurs.

BHATTI

CamScanner

day/date

TCP congestion control details:



cwnd: congestion window

This is a state variable maintained by the Sender.

This is NOT rwnd.

cwnd is ~~not~~ a sender's own variable & NOT send over the network.

cwnd is basically the total amount of MSS we can send at once

BHATTI

'date

cwnd is measured locally in the sender by measuring RTT, number of duplicate ACK's etc.

TCP follows this equation:

LastByteSent - LastByteACK ≤ cwnd.

basically:
sent but not ACK ≤ cwnd.

Flow:

1) A TCP sender transmits cwnd worth of bytes into the network

2) It pauses & waits one RTT for ACK of those bytes to return.

3) Upon receiving ACKs, the the LastByteACKed moves forward freeing up space to send more bytes.

BHATTI

CamScanner

day/date

Throughput equation:

$$\text{TCP rate} \approx \frac{\text{cwnd}}{\text{RTT}} \text{ bytes/second}$$

IMP:

- TCP for congestion control finds a value of cwnd using algorithms reno & Tahoe ~~cubic~~ & then keep congestion control using
  last byte sent - last Byte ACKed $\leq$ cwnd.

- TCP has both flow control (uses rwnd) & congestion control (uses cwnd).
- so actually:

$$\text{Max bytes in Flight} \leq \min(\text{cwnd}, \text{rwnd})$$

so this way we cater both flow control & congestion control.

BHATTI

CamScanner

'date

help with
TCP algos to ~~decide~~ cwnd:

1) TCP slow start:

- When connection begin start with cwnd = 1MSS & increase rate exponentially. UNTIL first loss event:

- Initially cwnd = 1MSS.
- After every RTT double cwnd. (i.e for every ACK double cwnd).

But then when should it stop?

e.g router can handle 33 packets currently cwnd = 32 & they all go but in next turn cwnd = 64 & we loose 31 packets!

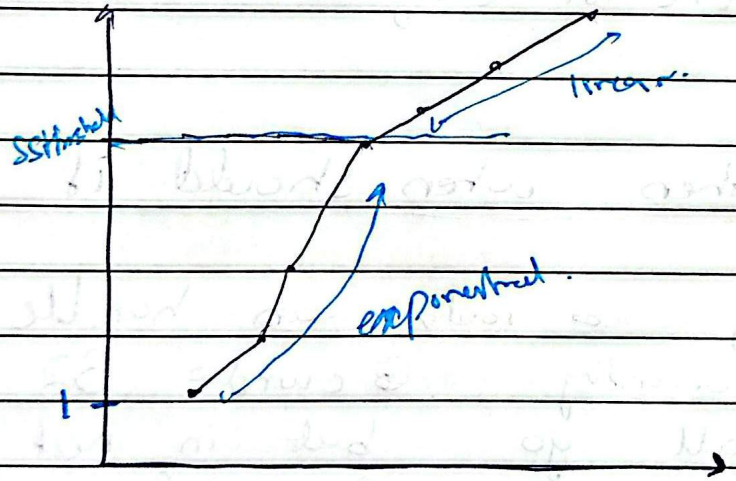
So we need to decide a threshold TO STOP the exponential increase.

BHATTI

day/date

What we do is we ~~expect~~ find a threshold when we know we are about to reach the maximum throughput point. After that instead of exponential increase, we do linear increase.

SS Threshold ← variable



BHATTI

CamScanner

e

At start, the ssthresh is set to a large number typically.

If loss happens.

$$ssthresh = \frac{1}{2} * cwnd.$$

What happens to cwnd at loss?

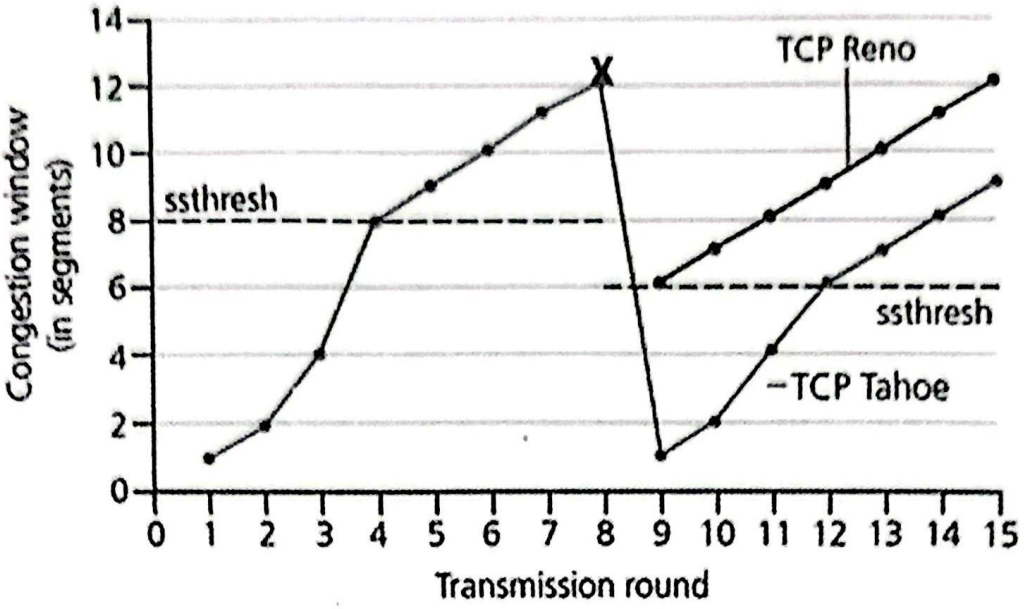
It totally depends on if TCP reno is implemented or Tahoe.

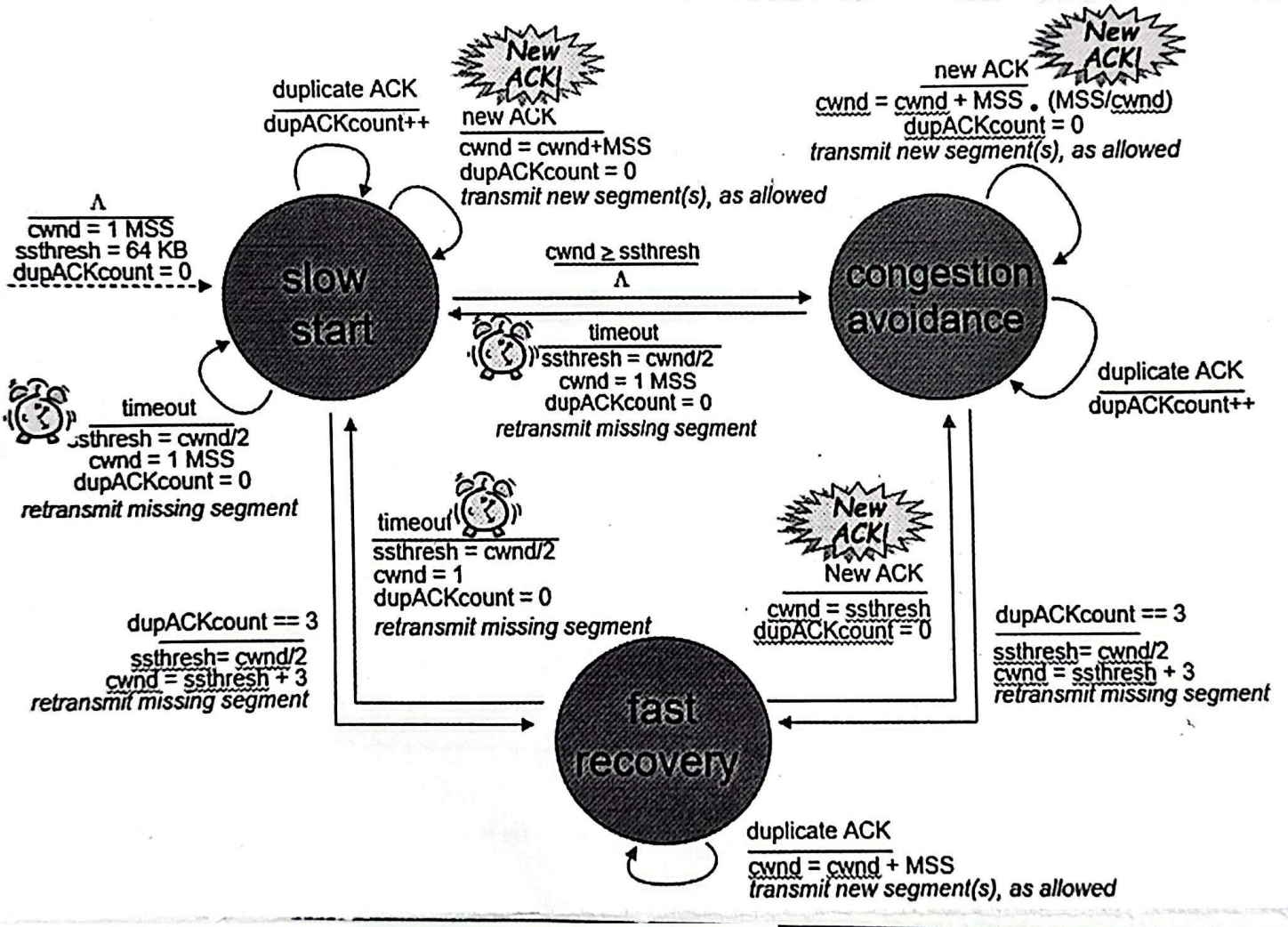
If Reno then $\frac{cwnd}{2}$ or cwnd = 1MSS

& if Tahoe then cwnd = 1 MSS.

BHATTI

CamScanner


Congestion window (in segments)
Transmission round
ssthresh
TCP Reno
TCP Tahoe
ssthresh
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
0 2 4 6 8 10 12 14

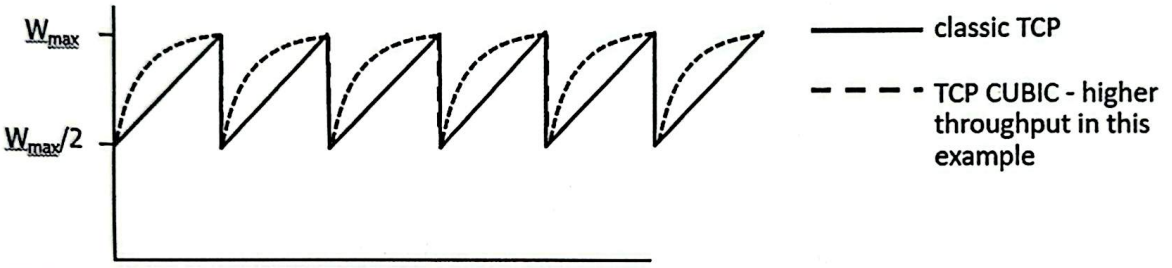

slow start
congestion avoidance
fast recovery
duplicate ACK
dupACKcount++
new ACK
cwnd = cwnd+MSS
dupACKcount = 0
transmit new segment(s), as allowed
Λ
cwnd = 1 MSS
ssthresh = 64 KB
dupACKcount = 0
cwnd ≥ ssthresh
Λ
timeout
ssthresh = cwnd/2
cwnd = 1 MSS
dupACKcount = 0
retransmit missing segment
new ACK
cwnd = cwnd + MSS • (MSS/cwnd)
dupACKcount = 0
transmit new segment(s), as allowed
duplicate ACK
dupACKcount++
timeout
ssthresh = cwnd/2
cwnd = 1 MSS
dupACKcount = 0
retransmit missing segment
timeout
ssthresh = cwnd/2
cwnd = 1
dupACKcount = 0
retransmit missing segment
New ACK
cwnd = ssthresh
dupACKcount = 0
dupACKcount == 3
ssthresh= cwnd/2
cwnd = ssthresh + 3
retransmit missing segment
dupACKcount == 3
ssthresh= cwnd/2
cwnd = ssthresh + 3
retransmit missing segment
duplicate ACK
cwnd = cwnd + MSS
transmit new segment(s), as allowed

CamScanner

late

2) TCP Cubic:

In this we look for a better way to reach maximum bandwith such that most of our data is sent at higher throughput



This tries to fix the the problem with AIMD (that uses Reno & Tahoe).

In this: Instead of growing our cwnd based on RTT or incoming ACKs, Cubic determines window growth based on elapsed time since last congestion event.

BHATTI

CamScanner

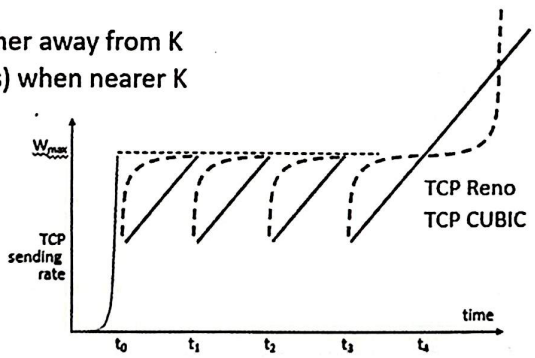
day/date

It's window size is calculated using a Cubic function:

$$W(t) = C \cdot (t - K)^3 + W_{max}$$

W_{max}: window size before loss.
t: The elapsed time since last loss of event.
C: A scaling constant
K: The time required to grow window back to W_{max} without any loss.

- K: point in time when TCP window size will reach W_{max}
  - K itself is tuneable
- increase W as a function of the *cube* of the distance between current time and K
  - larger increases when further away from K
  - smaller increases (cautious) when nearer K
- TCP CUBIC default in Linux, most popular TCP for popular Web servers



CamScanner

late

TCP & congested bottleneck Link:

TCP (reno, tahoe, Cubic) increase TCP's sending rate until packet loss occurs at some router's output: Bottleneck Link.

Out of 10's of router's in the path one would be the bottleneck because of lowest bandwidth / Throughput

For understanding congestion, why not focus on this bottleneck link.

[redacted]

BHATTI

CamScanner

day/date

Delay based TCP congestion control:

In this we keep track of the smallest RTT (RTT_{min}) over the connection's lifetime.

The RTT_{min} can represent the propogation delay when there is no intermediate queuing delay.

In each iteration we measure 2 Throughputs:

Expected Throughput (Best possible) = cwnd / RTT_{min}

Measured/Actual Throughput = cwnd / RTT_{actual}

If Actual Throughput is very close to Expected Throughput, increase cwnd. Because this means we are getting Throughput we want & can get more.

BHATTI

CamScanner

/date

If measured/Attained Throughput is far away from expected, then it shows Congestion so decrease cwnd linearly.

- congestion control without inducing/forcing loss
- maximizing throughout ("keeping the just pipe full… ") while keeping delay low ("…but not fuller")
- a number of deployed TCPs take a delay-based approach
  - BBR deployed on Google's (internal) backbone network

BHATTI

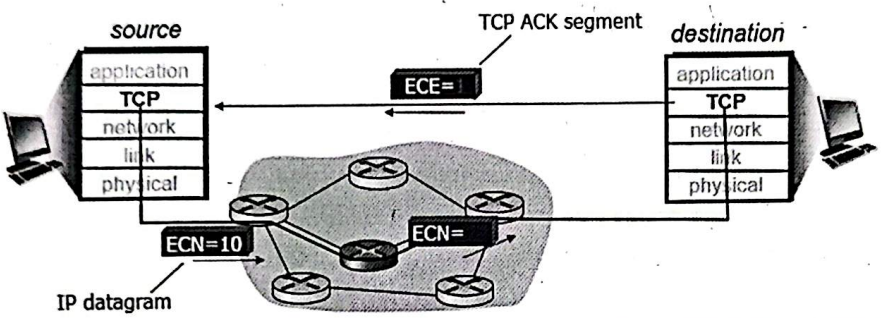
CamScanner

day/date

Explicit congestion Notification:

TCP deployments often implement *network-assisted* congestion control:

- two bits in IP header (ToS field) marked *by network router* to indicate congestion
  - *policy* to determine marking chosen by network operator
- congestion indication carried to destination
- destination sets ECE bit on ACK segment to notify sender of congestion
- involves both IP (IP header ECN bit marking) and TCP (TCP header C,E bit marking)



→ Router marks bits in network layer.

→ These are read by Destination which sets BCE bits in the TCP response to sender.

→ Then sender knows that congestion happened.

→ Involves both IP layer & TCP layer

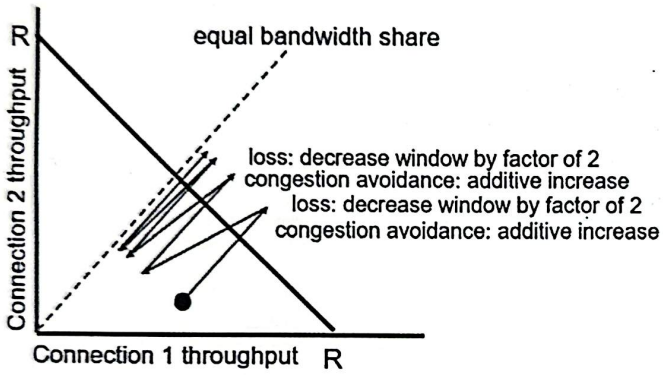
BHATTI

TCP Fairness:

IF k TCP sessions share some bottleneck link of bandwidth R, each should have avg rate of R/K.

Example: two competing TCP sessions:

- additive increase gives slope of 1, as throughout increases
- multiplicative decrease decreases throughput proportionally



Is TCP fair?

A: Yes, under idealized assumptions:

- same RTT
- fixed number of sessions only in congestion avoidance

Fairness: must all network apps be "fair"?

Fairness and UDP

- multimedia apps often do not use TCP
  - do not want rate throttled by congestion control
- instead use UDP:
  - send audio/video at constant rate, tolerate packet loss
- there is no "Internet police" policing use of congestion control

Fairness, parallel TCP connections

- application can open *multiple* parallel connections between two hosts
- web browsers do this , e.g., link of rate R with 9 existing connections:
  - new app asks for 1 TCP, gets rate R/10
  - new app asks for 11 TCPs, gets roughly R/2

CamScanner

day/date

Evolution of Transport layer functionality:

TCP & UDP have been the principle transport layers for 40 years & different TCP versions made for different changes:

Scenario	Challenges
Long, fat pipes (large data transfers)	Many packets "in flight"; loss shuts down pipeline
Wireless networks	Loss due to noisy wireless links, mobility; TCP treat this as congestion loss
Long-delay links	Extremely long RTTs
Data center networks	Latency sensitive
Background traffic flows	Low priority, "background" TCP flows

Now how about we move transport layer functions to APP layer & use UDP on Transport layer.

HTTP/3 : Quic

BHATTI

CamScanner

te

Quic: Quick UDP Internet Connections.

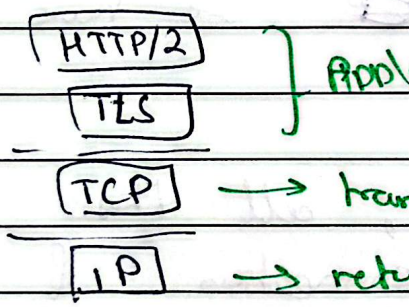
This is an applayer protocol build on top of UDP

- Instead of having security & reliability on transport layer it moves it to Applayer.

It offers:

→ error & congestion control
→ connection reliability.

In TCP:



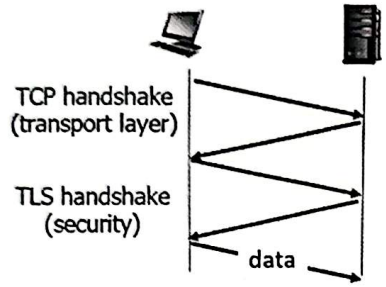
TLS & TCP both have different handshakes so it takes 2RTT before sending any data.

BHATTI

CamScanner

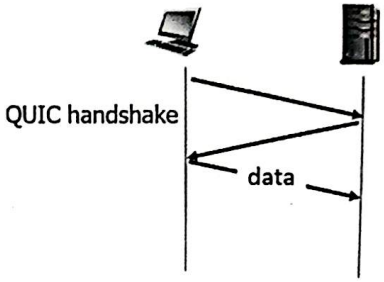
day/date

In Quic only one Handshake is needed.



TCP (reliability, congestion control state) + TLS (authentication, crypto state)

- 2 serial handshakes



QUIC: reliability, congestion control, authentication, crypto state

- 1 handshake

Quic: Parralled streaming, no HOL blocking.

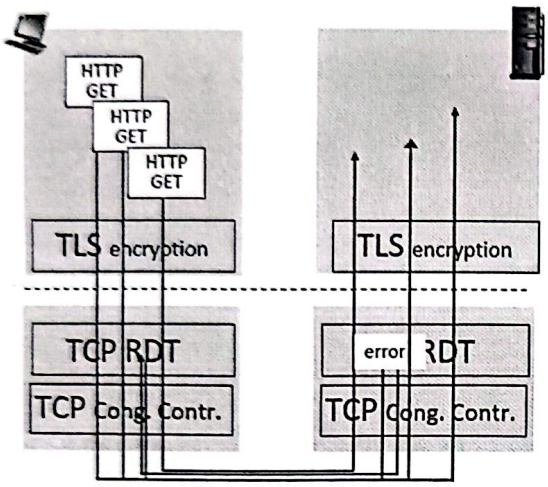
Consider ~~is~~ we have 3 web pages open.

In TCP, all these 3 open on a single connection & ~~the~~ packets of second page need to wait until first page packet sent. Because this in in transport layer.

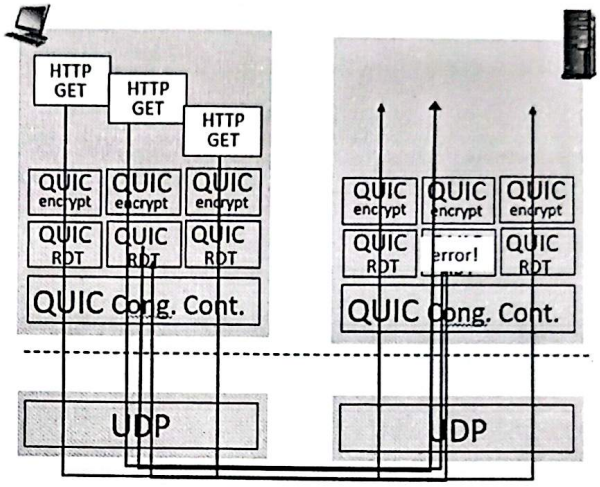
BHATTI

/date

But in Quic parallel streams can be opened for different pages. & even if one is blocked due to error others work.



(a) HTTP 1.1



(b) HTTP/2 with QUIC: no HOL blocking

BHATTI

CamScanner